

Backend Engineering Assignment

Context

You're joining a company that operates a **B2B e-commerce platform for agricultural and machinery parts**. The platform serves dealers and workshops across **20+ markets and languages** in Europe.

The platform consists of several domains:

- **Product Catalog** — millions of parts across thousands of categories
- **Pricing** — market-specific pricing, customer-specific discounts, and promotions
- **Availability** — real-time stock levels across multiple warehouses
- **Customer Context** — customer segments, purchase history, and preferences

Currently, when a customer views a product, the frontend makes **multiple separate API calls** to gather all relevant information. This causes:

- Slow page loads (especially on mobile connections in rural areas)
- Inconsistent behavior when one service is slow or unavailable
- Duplicated logic across different client applications (web, mobile, dealer tools)

The Architecture team wants to build a **Product Information Aggregator** — a backend service that combines data from multiple internal services into a single, market-aware response.

Your Task

Design and implement a backend service that aggregates product information for a given customer context.

Input: Product ID + Market/Language + Customer ID (optional)

Output: Aggregated product information suitable for display

Data Sources (to simulate)

You will create **mock implementations** of these upstream services:

Service	Returns	Typical Latency	Reliability
Catalog Service	Product details (name, description, specs, images)	50ms	99.9%
Pricing Service	Base price, customer discount, final price	80ms	99.5%
Availability Service	Stock level, warehouse location, expected delivery	100ms	98%
Customer Service	Customer segment, preferences (optional, only if customer ID provided)	60ms	99%

Important: Your mocks should behave realistically — include artificial delays and occasional failures based on the reliability percentages above.

Requirements

Functional

- Accept product ID, market code (e.g., "nl-NL", "de-DE", "pl-PL"), and optional customer ID
- Return aggregated product information
- Handle missing or unavailable data gracefully
- Response must be appropriate for the requested market/language

Non-Functional

- Service should respond within a reasonable time even when upstream services are slow
- Failure of one upstream service should not break the entire response
- Design should allow adding new data sources without major refactoring

Constraints

- If **Catalog Service** fails, the entire request should fail (we can't show a product without basic info)
- If **Pricing Service** fails, return the product but indicate price is unavailable
- If **Availability Service** fails, return the product but indicate stock is unknown
- If **Customer Service** fails (or no customer ID provided), return standard/non-personalized response
- REST endpoints are expected; *gRPC implementation is a bonus*

What We're Looking For

1. **Your approach to the problem** — How do you structure the aggregation? How do you handle partial failures? How do you deal with the "required vs optional" data distinction?
2. **Technical decisions** — Why did you choose certain patterns or libraries? What trade-offs did you consider?
3. **Production readiness** — How does your service handle failures? How would you operate it?
4. **Code quality** — Is your code clear, testable, and maintainable?

Deliverables

5. **Working code** in Java (17+), Kotlin, or Go
6. **README** explaining:
 - How to run the service
 - Your key design decisions and trade-offs
 - What you would do differently with more time
 - Answer to the Design Question below
7. **Be prepared to discuss** your solution in a 45-minute follow-up session

Design Question

In your README, briefly address ONE of these (your choice):

Option A: "The Assortment team wants to add a 'Related Products' service (200ms latency, 90% reliability). How would your design accommodate this? Should it be required or optional?"

Option B: "The Pricing team needs to support 'flash sales' where prices change every few minutes. How would this affect your design?"

Option C: "The platform expands to 5 new markets. What in your current design would need to change?"

You don't need to implement this — just explain your thinking.

Important Notes

- **There is no single "correct" solution.** We're interested in your reasoning.
- **You may use AI tools** (GitHub Copilot, ChatGPT, etc.) — we assume most developers do. Be prepared to explain every line of your code and defend your decisions.
- **Time investment:** We respect your time. Aim for 3-4 hours. A focused, well-reasoned solution is better than a feature-complete one.
- **The mocks matter.** How you simulate the upstream services tells us about your understanding of distributed systems.

Scenarios to Consider

During the follow-up session, we might explore scenarios like:

- *"A customer in Poland requests a product. Walk me through exactly what happens."*
- *"The Pricing Service starts timing out. What does the customer see?"*

Submission

- Share your code via a GitHub
- Ensure we can run it locally with minimal setup
- **Time expectation:** 3-4 hours should be sufficient. We value focused, well-reasoned solutions over feature-completeness.